

学习笔记

1. 调用函数库

```
import random, torch, matplotlib
import numpy as np
import pandas as pd
import torch.nn as nn
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
```

random: 引用随机数

torch: "神经网络机器人的工具包"——里面有搭骨架的零件、有训练它的方法、有保存它的工具。

pandas: 读入*csv*数据

numpy: 此题中转化为*tensor*使用, 因为*tensor*可以自动求导

matplotlib: 绘图工具

train_test_split: 将数据按比例随机划分为训练集和测试集

2. 随机数

```
seed = 214
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
```

使用地点:

- (1) 将数据按比例随机划分为训练集和测试集。
- (2) *Dataloader*将数据传入*train*时也需要随机。

3. 数据清洗和特征处理

```
df = pd.read_csv("data/train.csv")
```

先用*pandas*读取*csv*文件数据

```
y = df["Survived"].values
df = df.drop(columns = ["PassengerId", "Name", "Ticket", "Cabin", "Survived"])
```

将*Survived*作为标签, 即为我们预测值*y*

而特征值中的*PassengerID*, *Name*, *Ticket*, *Cabin*是字符串类型而*MLP*中一般使用*float*类型且这些数据对是否幸存无影响, 所以我们直接去除。也要将*Survived*从*df*中去除。

使用`DataLoader`每次从`Dataset`中取出`batch_size`组数据，训练集中数据需要随机取出。

5. MLP

```
class MLP(nn.Module):
    def __init__(self, in_dim, hd = 16):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hd),
            nn.ReLU(),
            nn.Linear(hd, hd // 2),
            nn.ReLU(),
            nn.Linear(hd // 2, 1)
        )
    def forward(self, x):
        return self.net(x)
```

`in_dim`表示每次输入进`MLP`中的数据量，`hd`为`hidden`的缩写即为隐藏层第一层的数据量。

我的设计是 $In_dim \rightarrow 16 \rightarrow 8 \rightarrow 1$ 总共4层。

其中`Linear`为线性函数 $y = w \cdot x + b$ 这`nn.Linear`即为加权求和。

其中 w 为权重， b 为偏置，这两个即为我们需通过机器学习调整的核心参数。

`ReLU`为简单激活函数， $\text{ReLU}(x) = \max(0, x)$ 。

6. 损失函数

```
loss_fn = nn.BCEWithLogitsLoss()
```

`BCELoss` 需要手动先 `sigmoid`。问题是：当 `logit` 很大或很小时，`sigmoid` 结果会非常接近 0 或 1，再取对数时会出现除以接近0的数，数值不稳定，容易出错。

`BCEWithLogitsLoss` 把 `sigmoid` + 交叉熵 两步合并成一个数学公式，绕开了这个不稳定的中间步骤，数值更稳，所以我们二分类使用它。

7. 优化器

```
opt = torch.optim.Adam(model.parameters(), lr = 1e-3)
```

`model.parameters()`：把模型里所有可学习的参数 (w 和 b) 交给优化器管理

$lr = 1e - 3$ ：学习率 = 0.001，每次更新的步长。

```
pred = model(xb).squeeze(1)
loss = loss_fn(pred, yb)
opt.zero_grad()
loss.backward()
opt.step()
```

优化器的作用：损失算出来、反向传播求出梯度后，优化器根据梯度去更新参数，让下一次预测更准。

`opt.zero_grad()`：清空梯度。

`loss.backward()`: 从`loss`出发回溯, 求出每个参数的梯度。

`opt.step()`: 更新参数。

所以我们只需循环这几步就可以了。

8. 训练过程中遇到的问题及解决方法:

1.需要学习的新函数与算法有点多, 刚开始一头雾水不知何处下手。

解决方法: 先让`ai`生成具体学习方案, 再通过b站具体研究每个函数与算法。

2.代码实现时遇到很多细节错误, 导致报错和算法错误。

解决方法: 学习`ai`的规范化写法, 再具体研究每处细节。

9. 心得体会:

预测的本质是通过设计函数并调整来拟合数据, 最终达到预测的效果。

`MLP`即将特征值即原始数据通过设计的函数转化为概率。再将此概率与标签`y`进行对比, 得到`loss`。

接着使用优化器不断调整函数参数, 达到提高预测准确率的效果。

10. 实验结果分析:

开始预测准确率会快速提高, 然后趋于平缓。

`loss`开始会快速下降, 然后趋于平缓, 会有小范围上下浮动。